UNIVERSIDADE FEDERAL DE SERGIPE

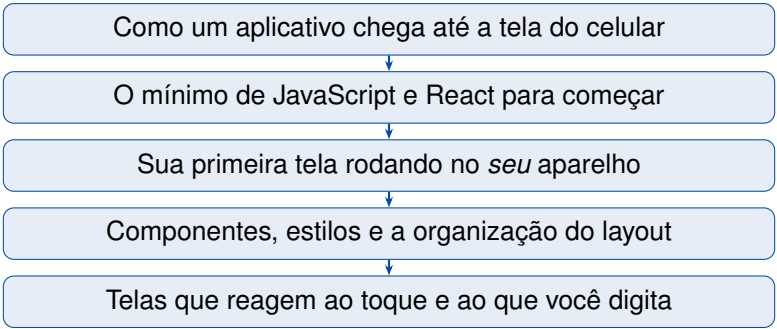
# Fundamentos de React Native

## E as suas primeiras telas rodando no celular

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

# O que você vai aprender hoje



# Onde a aula termina

## A meta de hoje

Daqui a 150 minutos o seu celular vai estar rodando um *cartão de perfil* escrito por você: foto, nome, bio editável, etiquetas de habilidades, um botão de curtir que conta e um botão de seguir que muda de cor.

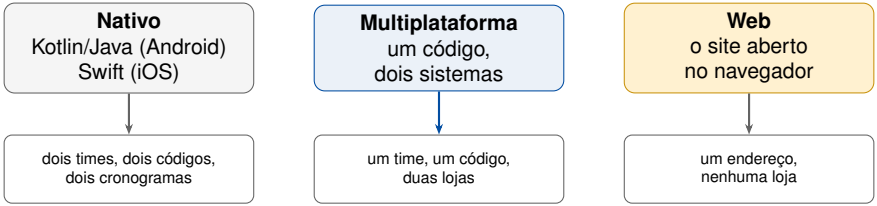
## O que você precisa

Um navegador, o celular com o aplicativo **Expo Go** instalado e a rede da universidade. Nada para compilar, nada para instalar no computador.



# Como um aplicativo chega ao celular

# Três caminhos para o mesmo aplicativo




A escolha não é sobre qual linguagem é mais bonita:
é sobre custo de manutenção, acesso ao hardware e velocidade de entrega.

Fonte: Eisenman (2017), cap. 1, “Advantages of React Native” [4]

# Nativo: o teto mais alto e a conta mais cara

**O que você ganha**

- desempenho e fluidez máximos;
- acesso imediato a todo recurso novo do sistema;
- ferramentas maduras de depuração e perfil.

**O que você paga**

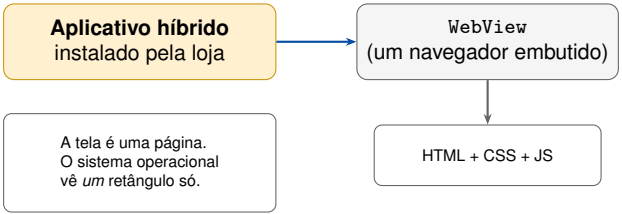
- escrever *duas vezes* a mesma tela;
- manter duas bases de código em sincronia;
- contratar duas especialidades distintas.

**Quando compensa**

Aplicativos que dependem de câmera, sensores ou gráficos em tempo real — e equipes grandes o bastante para bancar dois times.

Fonte: Eisenman (2017), cap. 1 [4]

# Web e híbrido: o mesmo HTML dentro de uma moldura




## A consequência que se sente no dedo

Rolagem, teclado e animações imitam o comportamento nativo em vez de *serem* o comportamento nativo — e a diferença aparece justamente nos gestos mais usados.

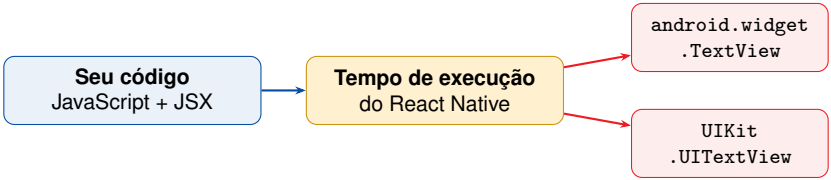
Fonte: Eisenman (2017), cap. 1, “Hybrid Applications” [4]

# Pergunta

Quando você escreve `<Text>Olá</Text>` em React Native, o que exatamente aparece na tela do celular?

- um <p> de HTML?
- um desenho em um *canvas*?
- um componente do sistema?

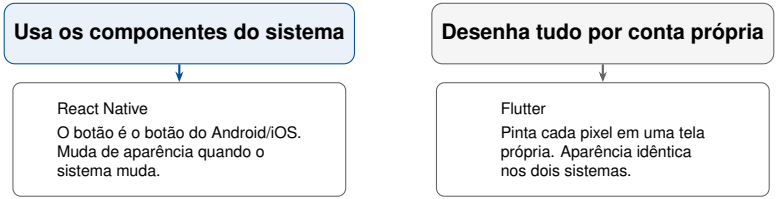
# Resposta: um componente nativo de verdade




O `<Text>` *não* é uma imitação: é o widget de texto do próprio sistema.
Por isso a seleção de texto, a rolagem e a acessibilidade já vêm prontas.

Fonte: React Native Docs, “Core Components and Native Components” [8]

# Duas famílias de multiplataforma




Nenhuma das duas é “a certa”. A pergunta é se você quer parecer com o sistema do usuário ou parecer sempre igual a si mesmo.

Fonte: React Native Docs, “Architecture Overview” [8]

# Recapitulando: o panorama

| | Nativo | React Native | Web / híbrido |
|---|---|---|---|
| Bases de código | duas | uma | uma |
| O que desenha a tela | o sistema | o sistema | o navegador |
| Acesso a sensores | total | amplo | limitado |
| Custo de manutenção | alto | médio | baixo |
| Distribuição | lojas | lojas | endereço web |

**A frase para levar**

React Native escreve-se como uma página e executa-se como um aplicativo: a sua lógica é JavaScript, mas a interface é feita de componentes nativos.

Fonte: Eisenman (2017), cap. 1 [4]; React Native Docs [8]

# JavaScript e React em trinta minutos

# Por que parar em JavaScript antes de abrir o editor

**O problema**

Quase todo erro da primeira semana de React Native não é de React Native: é de JavaScript. Uma chave fora do lugar, uma função que devolve `undefined`, um `map` sem `return`.

**O recorte de hoje**

Cinco construções, e só elas:

- *arrow functions*;
- desestruturação;
- `map`;
- JSX;
- componentes e *props*.

A ideia central: em JavaScript, funções são valores — dá para guardá-las em variáveis e passá-las como argumento.

Fonte: Flanagan (2020), cap. 8, “Functions” [6]

# Arrow functions: a função como valor

```
// forma tradicional: uma declaracao de funcao
function dobro(x) {
  return x * 2;
}

// a mesma funcao, escrita como valor guardado em uma constante
const dobro = (x) => x * 2;        // corpo curto: o return e' implicito

// quando o corpo tem mais de uma linha, volta o bloco e o return
const saudacao = (nome) => {
  const texto = 'Ola, ' + nome;
  return texto;
};
```

Fonte: Flanagan (2020), §8.1.3, “Arrow Functions” [6]

# Desestruturação: tirar campos de dentro de um objeto

**Desestruturação**: atribuição em que o lado esquerdo imita o formato do dado — cada variável recebe o campo de mesmo nome (objeto) ou a mesma posição (array).

## Sem desestruturação

```
const aluno = { nome: 'Ada', nota: 9.5 };
const nome = aluno.nome;
const nota = aluno.nota;
```

## Com desestruturação

```
const { nome, nota } = aluno;
// tambem funciona no parametro:
function resumo({ nome, nota }) {
  return nome + ': ' + nota;
}
// e em arrays, por posicao:
const [a, b] = [10, 20];
```

A forma de array é a que o `useState` usa: `const [valor, setValor] = useState(0)`.

Fonte: Flanagan (2020), §3.10.3, “Destructuring Assignment”, e §8.3.5 [6]

# `map`: de uma lista de dados para outra lista

```
const habilidades = ['JavaScript', 'React Native', 'Flexbox'];

// map percorre a lista, aplica a funcao e devolve uma NOVA lista
const tamanhos = habilidades.map((h) => h.length);
// tamanhos vale [10, 12, 7]; habilidades continua intacta

const marcadas = habilidades.map((h) => '#' + h.toLowerCase());
// ['#javascript', '#react native', '#flexbox']
```

## Por que isso importa em uma tela

Em React Native a lista de saída não é de textos: é de *elementos de interface*. Uma etiqueta na tela para cada habilidade no vetor.

Fonte: Flanagan (2020), §7.8.1, “`map()`” [6]

# Pergunta

```
const notas = [7, 5, 9, 4];

const resultado = notas.map((n) => {
  n >= 6;
});
```

Quanto vale `resultado`?

E o que seria preciso mudar para obter `[true, false, true, false]`?

# Resposta: o `return` que faltou

```
// resultado vale [undefined, undefined, undefined, undefined]
// o corpo em bloco { } executa a comparacao e joga fora o valor
```

```
// corrigindo com return
notas.map((n) => {
  return n >= 6;
});
```

```
// ou com corpo curto
notas.map((n) => n >= 6);
```

**A regra**

Chave depois da seta significa bloco de comandos, e bloco de comandos exige `return`. Sem chave, o valor da expressão já é o retorno.

Fonte: Flanagan (2020), §8.1.3 [6]

# O que é JSX

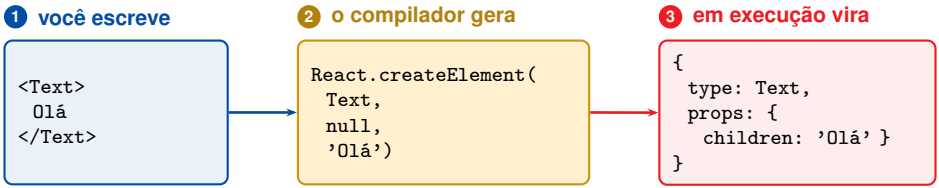
**JSX** (*JavaScript XML*) é uma **extensão de sintaxe** do JavaScript: tags no estilo XML/HTML, escritas no meio do código, que descrevem a tela. A forma do código imita a forma da interface.

- **Não é HTML**: as tags são componentes (`<View>`), não `<div>`;
- **Não é texto**: é uma *expressão*, um valor;
- **Não é JavaScript padrão**: o compilador Babel (embutido no Expo) traduz antes de rodar.

```
const nome = 'Ada';
// JSX e' um valor: vai numa variavel
const oi = <Text>Oi, {nome}</Text>;
// ... e num return
function Tela() {
  return <View>{oi}</View>;
}
```

Fonte: React Docs, “Writing Markup with JSX” [7]; Banks & Porcello (2020), cap. 5 [2]

# JSX por baixo: cada tag vira uma chamada de função




JSX é **açúcar sintático** para chamadas de função.
O celular nunca vê a marcação: vê o objeto que ela produz.

Fonte: Banks & Porcello (2020), cap. 5, “React with JSX” [2]

# Quatro regras de JSX que sempre pegam

```
// 1. chaves entram em modo JavaScript
<Text>Total: {2 + 3}</Text>

// 2. so' pode haver UM elemento raiz
<View>
  <Text>a</Text>
  <Text>b</Text>
</View>

// 3. toda tag fecha, inclusive as vazias
<Image source={foto} />
```

```
// 4. estilo e' objeto, nao string
<Text style={{ fontSize: 20 }}>
  Ola
</Text>

// as duas chaves: a de fora abre o
// modo JavaScript, a de dentro e'
// o literal de objeto
```

Nomes de propriedade seguem JavaScript: `background-color` vira `backgroundColor`.

Fonte: Banks & Porcello (2020), cap. 5 [2]

# Componente: uma função que devolve JSX

```
// componente = funcao com nome MAIUSCULO que devolve JSX
function Saudacao() {
  return <Text>Bom dia!</Text>;
}

// usar o componente e' escrever uma tag com o nome dele
function App() {
  return (
    <View>
      <Saudacao />
      <Saudacao />
    </View>
  );
}
```

Fonte: React Docs, “Your First Component” [7]

# Props: os parâmetros do componente

### Quem recebe

```
// as props chegam como UM objeto;
// a desestruturacao abre esse objeto
function Saudacao({ nome, horario }) {
  return <Text>Bom {horario}, {nome}!</Text>;
}
```

### Quem envia

```
<View>
  <Saudacao nome="Ada" horario="dia" />
  <Saudacao nome="Alan" horario="tarde" />
</View>
```

**Props são somente leitura**

Quem decide o valor é quem chamou o componente. Para mudar algo de dentro, é preciso *estado* — o último bloco da aula.

Fonte: React Docs, “Passing Props to a Component” [7]

# Recapitulando: JavaScript e React

`(x) => x * 2` — função como valor; sem chave, o retorno é implícito

`const { nome } = aluno` — abre o objeto; é assim que props chegam

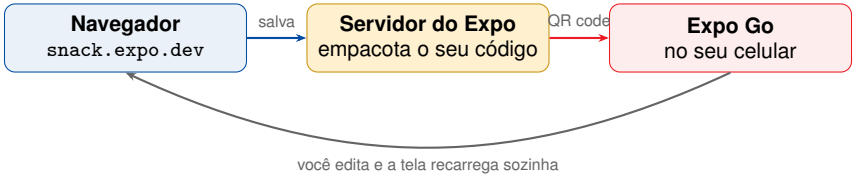
`lista.map(...)` — uma lista de dados vira uma lista de elementos

`<View><Text/></View>` — JSX: um elemento raiz, tudo fechado

`function Cartao({ nome })` — componente com nome maiúsculo e props

# A primeira tela no seu celular

# Por que Snack e Expo Go, e não a instalação completa




## O que isso nos poupa hoje

Android Studio, Xcode, SDK, emulador e JDK — somados, várias horas de instalação e um disco cheio. Nada disso é necessário para as ideias de hoje.

Fonte: Expo Docs, “Snack” e “Expo Go” [5]

# Cinco passos, cinco minutos

1. No celular, instale o **Expo Go** (Play Store ou App Store).
2. No computador, abra `snack.expo.dev` em uma aba nova.
3. Na barra direita, escolha **My Device**: aparece um QR code.
4. Abra o Expo Go e leia o QR code (no iPhone, use a câmera).
5. Apague o conteúdo de `App.js` e cole o código do próximo slide.

**Se a rede da universidade bloquear a conexão**

Use o botão **Web** do próprio Snack: a tela roda no navegador. Perde-se o celular de verdade, mas nenhum conceito da aula.

Fonte: Expo Docs, “Get started” [5]

# O “Olá mundo” que vamos rodar

```
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={estilos.tela}>
      <Text style={estilos.titulo}>Ola, COMP0511!</Text>
    </View>
  );
}

const estilos = StyleSheet.create({
  tela: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  titulo: { fontSize: 28, fontWeight: 'bold' },
});
```

Fonte: React Native Docs, “Core Components” [8]

# Faça agora — 5 minutos

**Tarefa 1**

1. Ponha o “Olá mundo” para rodar no seu aparelho.
2. Troque o texto pelo seu próprio nome.
3. Mude `fontSize` para `48` e veja a tela recarregar sozinha.
4. Apague a linha `flex: 1` e observe para onde o texto vai.

**Se travar**
Chame; quem já conseguiu ajuda o vizinho. Ninguém segue para o próximo bloco sem ter visto o próprio nome na tela.

**Guarde o endereço**
O Snack gera uma URL. Salve-a: é nela que o cartão de perfil vai crescer pelo resto da aula.

# Quando não funciona: os quatro suspeitos de sempre

| Sintoma | Causa provável e o que fazer |
|---|---|
| O QR code não conecta | Celular e computador em redes diferentes. Use `Web` ou troque a rede. |
| Tela branca, sem erro | Faltou o `export default` antes de `function App`. |
| `Text strings must be rendered within a <Text>` | Texto solto dentro de `<View>`. Envolva-o em `<Text>`. |
| A imagem não aparece | `Image` com `uri` precisa de `width` e `height` no estilo. |

Fonte: React Native Docs, “Troubleshooting” [8]

# Componentes, estilos e layout

# O catálogo mínimo: cinco componentes bastam hoje

| Componente | Equivale a | Para que serve |
|---|---|---|
| `View` | `<div>` | caixa: agrupa, posiciona, recebe cor e borda |
| `Text` | `<p>` | todo texto visível; nada de texto solto |
| `Image` | `<img>` | imagem local ou vinda de uma URL |
| `TextInput` | `<input>` | entrada de texto do usuário |
| `Pressable` | `<button>` | qualquer área que reage ao toque |

**A regra que o iniciante mais esquece**

Texto só existe dentro de `<Text>`. Escrever palavras diretamente dentro de uma `<View>` é erro de execução, não aviso.

Fonte: React Native Docs, “Core Components and APIs” [8]

# Os cinco em uma tela só

```
<View style={estilos.tela}>
  <Text style={estilos.titulo}>Catalogo</Text>

  <Image style={estilos.foto}
         source={{ uri: 'https://picsum.photos/id/1005/200/200' }} />

  <TextInput style={estilos.campo} placeholder="Digite seu nome" />

  <Pressable style={estilos.botao} onPress={() => console.log('toquei')}>
    <Text style={estilos.textoBotao}>Tocar aqui</Text>
  </Pressable>
</View>
```

`Pressable` não desenha nada: ele envolve o que deve ser tocável.

Fonte: React Native Docs, “Pressable” e “Image” [8]

# `StyleSheet.create`: estilo separado da estrutura

### Inline: rápido e bagunçado

```
<Text style={{ fontSize: 20,
               color: 'blue' }}>
  Ola
</Text>
```

Um objeto novo a cada renderização; a estrutura some no meio da formatação.

### Com `StyleSheet`

```
<Text style={estilos.titulo}>Ola</Text>

const estilos = StyleSheet.create({
  titulo: { fontSize: 20,
            color: 'blue' },
});
```

Nomes com significado, reaproveitáveis, e o JSX volta a se ler como estrutura.

### Combinando estilos

`style={[estilos.botao, estilos.botaoAtivo]}` — o vetor aplica da esquerda para a direita; o último vence nos campos repetidos.

Fonte: React Native Docs, “Style” e “StyleSheet” [8]

# Pergunta

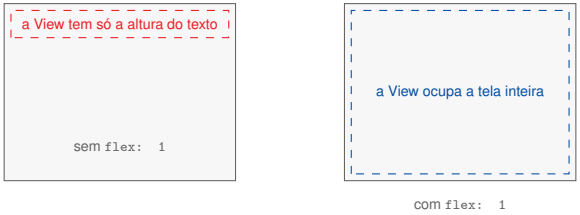
```
const estilos = StyleSheet.create({
  tela: { justifyContent: 'center', alignItems: 'center' },
});
```

A tela tem um único `<Text>` e o estilo acima.

Por que o texto continua colado no topo?

# Resposta: centralizar dentro de quê?

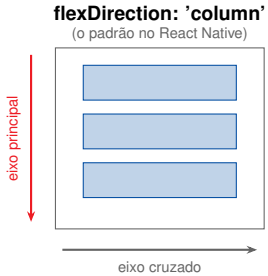
```
tela: { flex: 1, justifyContent: 'center', alignItems: 'center' }
```

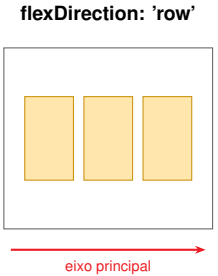



`justifyContent` distribui o conteúdo *no espaço que a caixa tem*. Se a caixa encolheu até caber no conteúdo, não há espaço para distribuir.

Fonte: React Native Docs, “Layout with Flexbox” [8]

# Flexbox: três propriedades resolvem quase tudo




`flexDirection`
escolhe o eixo principal

`justifyContent`
distribui no eixo principal

`alignItems`
alinha no eixo cruzado

Fonte: W3C, CSS Flexible Box Layout L1, §5–6 [1]; React Native Docs [8]

# Flexbox no celular não é igual ao da web

| | CSS na web | React Native |
|---|---|---|
| `flexDirection` padrão | `row` | `column` |
| `flex` | três valores | um número só |
| Unidades | `px`, `em`, `%` | número puro (pontos) |
| Herança de fonte | filhos herdam | cada `Text` tem o seu |
| Nome das propriedades | `background-color` | `backgroundColor` |

**Por que `column` é o padrão**

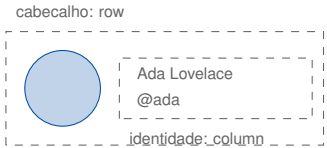
Telas de celular são altas e estreitas: empilhar é o caso comum. A web herdou do texto, que corre na horizontal.

Fonte: React Native Docs, “Layout with Flexbox” [8]; Eisenman (2017), cap. 5 [4]

# Exemplo integrado: o cabeçalho do cartão

```
<View style={estilos.cabecalho}>
  <Image style={estilos.foto}
         source={{ uri: PERFIL.foto }} />
  <View style={estilos.identidade}>
    <Text style={estilos.nome}>Ada Lovelace</Text>
    <Text style={estilos.usuario}>@ada</Text>
  </View>
</View>

cabecalho: { flexDirection: 'row',
             alignItems: 'center' },
foto:      { width: 72, height: 72,
             borderRadius: 36 },
identidade:{ marginLeft: 14 },
```




Uma `View` em linha, contendo uma `View` em coluna: é assim que quase todo cartão de rede social é montado.

# Recapitulando: componentes e layout

## O que aprendemos

- cinco componentes cobrem a tela de hoje, e texto só vive dentro de `Text`;
- `StyleSheet.create` devolve o JSX à sua função de descrever estrutura;
- `flex: 1` é o que dá à caixa o espaço que ela vai distribuir;
- `flexDirection` escolhe o eixo; `justifyContent` e `alignItems` posicionam nele e no eixo cruzado.

## Ainda falta uma coisa

Tudo o que fizemos até aqui é uma fotografia: bonita e imóvel. O botão não conta, o campo não lembra.

# Telas que reagem: `useState`

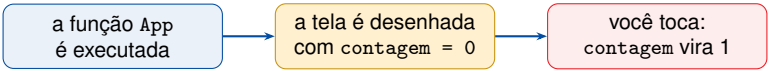
# Pergunta

```
export default function App() {
  let contagem = 0;

  return (
    <View>
      <Text>{contagem}</Text>
      <Pressable onPress={() => { contagem = contagem + 1; }}>
        <Text>somar 1</Text>
      </Pressable>
    </View>
  );
}
```

Você toca cinco vezes no botão. O que a tela mostra?

# Resposta: zero, sempre zero




A variável mudou na memória — e ninguém pediu para redesenhar a tela.
Pior: se algo redesenhar, a função roda de novo e `contagem` volta a valer 0.

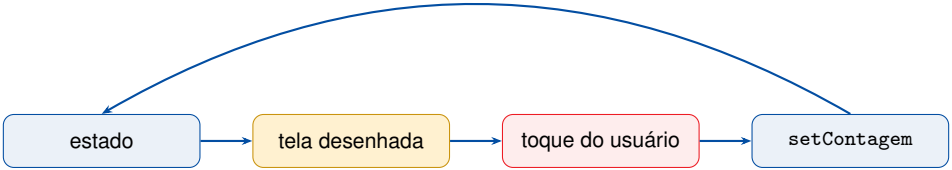
Falta um mecanismo que faça duas coisas ao mesmo tempo:
*lembrar* o valor entre as renderizações e *avisar* que a tela mudou.

Fonte: React Docs, “State: A Component’s Memory” [7]

# `useState`: memória e aviso na mesma linha

```
import { useState } from 'react';

const [contagem, setContagem] = useState(0);
//     |            |                    +-- valor inicial (so' na 1a vez)
//     |            +----------- troca o valor E pede um novo desenho
//     +------------------------ valor atual, valido nesta renderizacao
```




Fonte: React Docs, “State: A Component’s Memory” [7]; Banks & Porcello (2020), cap. 6 [2]

# Contador: o programa inteiro

```
import { useState } from 'react';
import { View, Text, Pressable, StyleSheet } from 'react-native';

export default function App() {
  const [contagem, setContagem] = useState(0);

  return (
    <View style={estilos.tela}>
      <Text style={estilos.numero}>{contagem}</Text>
      <Pressable style={estilos.botao} onPress={() => setContagem(contagem + 1)}>
        <Text style={estilos.textoBotao}>+1</Text>
      </Pressable>
    </View>
  );
}
```

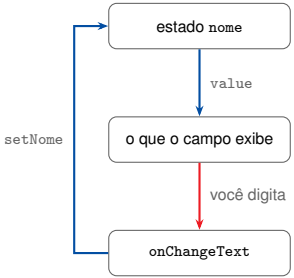
Fonte: React Native Docs, “Using Hooks” [8]

# Campo controlado: o estado manda no que aparece

```
const [nome, setNome] = useState('');

<TextInput
  value={nome}
  onChangeText={setNome}
  placeholder="digite aqui"
/>

<Text>
  {nome.length === 0
    ? 'Ainda nao digitou nada.'
    : 'Ola, ' + nome + '!'}
</Text>
```




O campo não guarda nada: mostra o estado e avisa a mudança.

Fonte: React Native Docs, “Handling Text Input” [8]

# Três armadilhas do estado

## 1. `set` não é atribuição imediata

```
setContagem(contagem + 1);
console.log(contagem);  // ainda o valor ANTIGO
```

## 2. Se o novo valor depende do antigo, use a forma funcional

```
setContagem((n) => n + 1);
```

## 3. Nunca altere o valor por dentro

```
lista.push('novo');            // ERRADO: React nao percebe
setLista([...lista, 'novo']);  // CERTO: um vetor novo
```

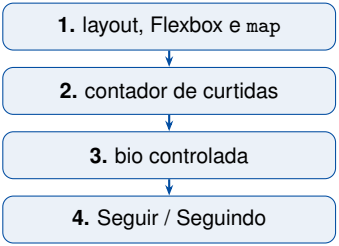
Fonte: React Docs, “Updating Arrays in State” [7]

# Tarefa 2 — o cartão de perfil é seu

**O que construir**

- cabeçalho com foto redonda, nome e usuário;
- bio editável por um campo controlado;
- etiquetas de habilidades geradas com `map`;
- botão de curtir que conta;
- botão Seguir/Seguindo que troca texto e cor.

**Como trabalhar:** em duplas, no mesmo Snack, a partir do esqueleto.



# O que cada checkpoint cobra

| # | Entrega | Conceito que ele verifica |
|---|---|---|
| 1 | cartão estático completo | `flexDirection: 'row'`; `map` com `key` |
| 2 | botão de curtir contando | `useState` numérico e a forma funcional do `set` |
| 3 | bio que muda enquanto se digita | `value` + `onChangeText` no mesmo estado |
| 4 | Seguir vira Seguindo | estado booleano decidindo texto, estilo e um valor derivado |

**Checkpoint 4, a pegadinha boa**

O total de seguidores *não* é estado novo: calcula-se a partir de `seguindo`. Estado a mais é fonte de inconsistência.

Fonte: React Docs, “Choosing the State Structure” [7]

# Fechamento

# O caminho que percorremos

React Native escreve-se como uma página e executa componentes **nativos**

arrow functions, desestruturação e `map` — o JavaScript que a tela exige

JSX vira chamada de função; componente é função com nome maiúsculo

`flex: 1` dá o espaço; `flexDirection` escolhe o eixo

`useState` é memória *e* aviso: sem ele a tela não se redesenha

# Para casa

**Tarefa**

Transforme o cartão de perfil em um **cartão de contato** com uma lista de contatos:

1. extraia um componente `Cartao` que receba `nome`, `usuario`, `foto` e `habilidades` por props;
2. monte um vetor com **quatro** pessoas e renderize os quatro cartões com `map`;
3. cada cartão mantém o próprio contador de curtidas (curtir um não pode afetar os outros);
4. acrescente um `TextInput` no topo que filtra os cartões pelo nome.

**Dica do item 3:** onde deve morar o estado das curtidas — no `App` ou dentro do `Cartao`?

# Na próxima aula

**De onde partimos**

Uma tela só, com os dados escritos no próprio arquivo. Quatro cartões já deixam a tela sem espaço para rolar.

**Para onde vamos**

- listas longas com `FlatList`;
- `ScrollView` e áreas seguras;
- navegação entre telas;
- dados vindos da rede.

**Antes da próxima aula**

Deixe o Expo Go instalado e o Snack de hoje salvo — vamos continuar exatamente deste ponto.

# Referências I



Tab Atkins Jr., Elika J. Etemad, and Rossen Atanassov.
CSS flexible box layout module level 1.
W3C Candidate Recommendation,
https://www.w3.org/TR/css-flexbox-1/, 2018.


Alex Banks and Eve Porcello.
*Learning React: Modern Patterns for Developing React Apps*.
O'Reilly Media, Sebastopol, CA, 2 edition, 2020.


Ecma International.
ECMAScript 2024 language specification (ECMA-262, 15th edition).
https://262.ecma-international.org/15.0/, 2024.

# Referências II


- Bonnie Eisenman.
  *Learning React Native: Building Native Mobile Apps with JavaScript*.
  O'Reilly Media, Sebastopol, CA, 2 edition, 2017.
- Expo.
  Expo documentation — snack and expo go.
  `https://docs.expo.dev/`, 2025.
  Acesso em 2026.
- David Flanagan.
  *JavaScript: The Definitive Guide*.
  O'Reilly Media, Sebastopol, CA, 7 edition, 2020.

# Referências III



Meta Platforms.
React documentation — learn react.
https://react.dev/learn, 2025.
Acesso em 2026.


Meta Platforms.
React native documentation.
https://reactnative.dev/docs/getting-started, 2025.
Acesso em 2026.